

Lightweight Integrity Auditing for IoT Cloud Data Based on Lattice Hash and Smart Contract

Hang Li

School of Engineering
Nanfang College, Guangzhou
Guangzhou, China
lih@nfu.edu.cn

Kaixin Wu

School of Engineering
Nanfang College, Guangzhou
Guangzhou, China
wukx230017@stu.nfu.edu.cn

Haibo Luo

School of Engineering
Nanfang College, Guangzhou
Guangzhou, China
luohb@nfu.edu.cn

Junying Yuan

School of Engineering
Nanfang College, Guangzhou
Guangzhou, China
yuanjy@nfu.edu.cn

Xin Ge*

School of Engineering
Nanfang College, Guangzhou
Guangzhou, China
*Gex@nfu.edu.cn

Abstract—With the deep integration of the Internet of Vehicles (IoV) and autonomous driving, AI-driven real-time decision-making systems impose higher requirements on the integrity and trustworthiness of in-vehicle data. In the context of “Intelligent Innovation and Ethical Co-governance”, data integrity involves core issues such as algorithmic fairness, responsibility attribution, and technological ethics. However, existing cloud data integrity auditing schemes suffer from three key problems: reliance on complex structures like Merkle trees leads to computational overhead that grows with data size; auditing depends on third parties, making results difficult to hold accountable; and the lack of transparent recording mechanisms fails to meet the “explainability and accountability” demands of AI ethics. To address these problems, this paper proposes a lightweight auditing scheme based on lattice hash and smart contracts. Specifically, the additive homomorphism of lattice hash eliminates tree structures and reduces computational overhead. Signed audit results are stored on the blockchain, enabling auditor accountability. Smart contracts are used for tamper-proof evidence storage without secondary verification, ensuring openness and transparency. Theoretical analysis and experiments demonstrate that our scheme not only ensures data confidentiality and integrity but also significantly reduces computational overhead while achieving publicly traceable audits and multi-party accountability. This work provides an efficient and ethically governed integrity auditing method for IoV cloud storage.

Keywords—cloud data integrity, lattice hash, smart contract

I. INTRODUCTION

With the deep integration of artificial intelligence and IoV technologies, intelligent transportation systems have put forward higher requirements for real-time data-driven decision-making capabilities. Modern vehicles can generate terabytes of data every day, including GPS trajectories, in-vehicle sensor readings, driving behavior logs, etc. [1]. These data are not only key inputs for autonomous driving algorithm training and real-time decision-making, but also important basis for traffic management, accident analysis, and insurance claims. However, vehicles have limited local storage and computing resources, so outsourcing data to cloud servers has become a common practice [2]. Although data outsourcing brings convenience, it

also introduces severe security and ethical challenges. Cloud servers are not completely trusted, and may delete or tamper with user data due to software and hardware failures, external attacks, or economic interests [3]. Once corrupted data is used by AI decision-making systems, it may cause serious traffic accidents. Moreover, responsibility attribution after data damage is extremely difficult: car owners can hardly prove that the damage occurred during cloud storage, while cloud service providers may also evade responsibility.

Existing schemes suffer from three major deficiencies. Firstly, the reliance on complex structures such as Merkle trees results in computational overhead that increases with the volume of data, making this approach unsuitable for resource-constrained in-vehicle environments. Second, auditing depends on third parties without effective constraints on auditor behavior, making the results difficult to hold accountable. Third, the lack of open and transparent evidence storage mechanisms fails to meet the traceability and accountability requirements of AI ethics.

To address these deficiencies, this paper proposes a lightweight auditing scheme based on lattice hash and smart contracts. The specific contributions are as follows:

- To tackle computational overhead, the additive homomorphism of lattice hash is leveraged for proof aggregation, significantly reducing the computational burden on both the cloud server and the auditor.
- To address auditor accountability, signed audit results are stored on the blockchain, establishing a "who audits, who is responsible" accountability chain.
- To ensure transparent evidence storage, smart contracts are used for tamper-proof recording without secondary verification, guaranteeing that audit records remain open, transparent, and immutable.

II. RELATED WORK

Classic data integrity verification schemes: The provable data possession (PDP) model proposed by Ateniese et al. [4]

and the proof of retrievability (PoR) scheme proposed by Juels et al. [5] laid the foundation for cloud data integrity verification. These schemes storage overhead to increase linearly with data volume. Moreover, the verification process involves multiple cryptographic operations, making it difficult to apply directly to vehicle terminals with limited computing power. To reduce the burden on users, Wang et al. introduced a Third-Party Auditor (TPA) to perform auditing tasks on behalf of users [6]. However, the TPA may forge audit results or collude with cloud servers, leaving users without effective means of accountability. In recent years, blockchain and smart contracts have been used to improve audit transparency. For instance, Liu et al. proposed an audit scheme based on a quad Merkle tree and blockchain [7]. Nevertheless, such schemes often require the smart contract to re-verify the proof or store a large number of tags on-chain, leading to double verification overhead and contradicting the lightweight nature of blockchain. Regarding lattice hash functions and IoV auditing, the lattice hash function proposed by Bellare and Micciancio features additive homomorphism and strong collision resistance [8], making it suitable for data aggregation verification. In the context of IoV auditing, Li et al. designed an in-vehicle digital forensics scheme supporting public auditing [9], and Tian et al. proposed an identity-based cloud storage auditing protocol for IoV data [10]. However, these schemes still fail to address the issues of transparent storage of audit results and accountability.

In summary, existing schemes involve trade-offs among efficiency, trust assumptions, and accountability mechanisms. This paper combines the homomorphic aggregation capability of lattice hashing with the transparent evidence storage advantage of smart contracts, striving to achieve both high efficiency and strong accountability in IoV scenarios..

III. SYSTEM MODEL AND DESIGN GOALS

A. Scenario Overview

Consider a typical IoV scenario: tens of thousands of vehicles continuously generate data during driving, such as location trajectories, vehicle speeds, sensor readings, driving behavior logs, etc. These data are of great value for intelligent traffic scheduling, accident analysis, insurance claims, etc. However, vehicle users expect to verify periodically whether their cloud data remains intact and untampered, even when the data is not stored locally. To this end, we propose a lightweight auditing framework based on lattice hash, and use smart contracts to audit results on the blockchain to realize the transparency and accountability of the audit process.

B. Entity Definition

This scheme includes the following four types of participants:

- **Vehicle:** that is, the data owner. The vehicle uploads the ciphertext to the cloud, and sends the tag to the auditor. The vehicle does not participate in the online audit process, and only needs to query the historical audit results from the blockchain when necessary.
- **Cloud Server (CS):** Responsible for storing the data uploaded by the vehicle. CS may cause data damage or loss due to software errors, hardware failures or

malicious attacks, but will honestly execute the challenge-response protocol during the audit process.

- **Auditor:** A semi-trusted third-party service. The auditor regularly (or on demand) initiates random challenges to the CS, receives the proof returned by the CS, and verifies the integrity of the data through bilinear mapping. The auditor signs each round of audit results (pass/fail) together with relevant metadata (timestamp, challenge index, proof hash, etc.) and sends them to the smart contract.
- **Smart Contract & Blockchain:** The smart contract verifies the validity of the signature sent by the auditor, and then writes the results into the persistent storage of the blockchain. As a public, immutable ledger, any vehicle or third party can query the historical audit records of a certain file. The system flow chart is shown in Fig. 1.

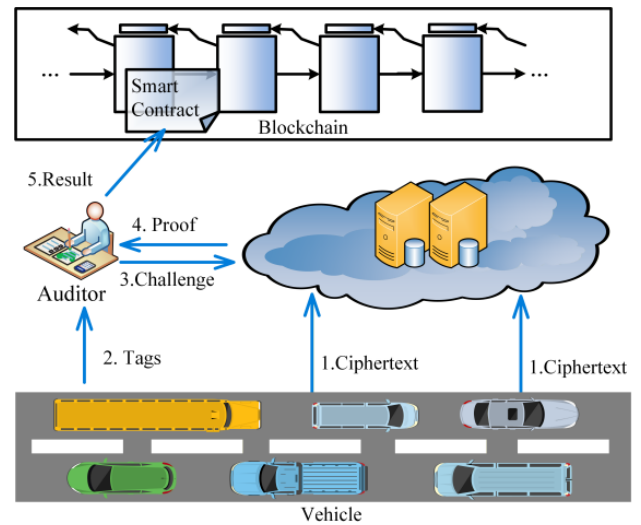


Fig. 1. System Architecture

C. Program details

1) Initialization Phase

First, a trusted third party (IoV service provider) generates global public parameters $Para$ and distributes them to all entities:

- Select a large prime number q .
- Define two multiplicative cyclic groups G and G_T of order q , and a bilinear map $e: G \times G \rightarrow G_T$.
- Select g as a generator of group G .
- Select a lattice hash function $H: \{0,1\}^* \rightarrow Z_q$ (or mapped to the additive representation of group G), which satisfies additive homomorphism: $H(m_1) + H(m_2) = H(m_1 || m_2)$ and has collision resistance. The specific construction of lattice hash can refer to LtHash. In this scheme, we use its homomorphism on the integer additive group.

Public parameters: $Para = \{q, G, G_T, g, e, H\}$.

The vehicle has the original file F (such as driving trajectory records), and performs the following steps:

- Symmetric encryption: The vehicle randomly generates a symmetric key SK , and uses a symmetric encryption algorithm (AES-256) to encrypt the file F to obtain the ciphertext $C = Enc_{SK}(F)$.
- Blocking: Since in subsequent audits, the auditor only randomly challenges part of the blocks instead of the entire file, thereby reducing communication and computing overhead. Blocking also facilitates the localization of partial corruption. Therefore, we divide the ciphertext C into n fixed-size blocks: $C = \{c_1, c_2, \dots, c_n\}$.

The vehicle generates a lightweight tag for the entire ciphertext file, which will be used for subsequent integrity verification: Select random number: The vehicle randomly selects a secret value $r \in Z_q$ (q is a large prime number). Calculate basic tag: g^r . Calculate the lattice hash value of the complete ciphertext: First calculate the lattice hash of the entire ciphertext $\{H(c_1), H(c_2), \dots, H(c_n)\}$, where H is the lattice hash function. Then calculate the file tag: $g^{r \cdot H(c_i)}$. Finally, the vehicle obtains the tag set of the file: $Tag = (g^r, g^{r \cdot H(c_1)}, g^{r \cdot H(c_2)}, \dots, g^{r \cdot H(c_i)})$.

The vehicle sends the generated data to different entities:

- Send to CS: The vehicle uploads the ciphertext blocks $\{c_1, c_2, \dots, c_n\}$ to CS. CS stores these blocks according to the index and returns the storage address of each block. CS does not store any tags, nor does it hold the decryption key SK .
- Send to auditor: The vehicle sends the tag set Tag to the auditor. The auditor stores Tag by file ID for all subsequent audit challenges.
- Local deletion by vehicle: The vehicle deletes the local ciphertext and tags, and only retains the symmetric key SK and basic file information (for future download and decryption). The vehicle does not need to participate in the online audit process.

2) Challenge Phase

The auditor regularly or irregularly initiates random challenges to the CS. CS uses the homomorphism of lattice hash to quickly aggregate the proof and return it. The auditor verifies the validity of the proof locally.

The auditor adopts probabilistic sampling auditing method, and randomly selects c different indexes from $\{1, 2, \dots, n\}$ to form the challenge set $Q = \{i_1, i_2, \dots, i_c\}$, where c is a preset parameter used to balance the detection probability and overhead.

To prevent replay attacks, the auditor generates a one-time random number nonce (a 32-byte random string) and binds it to the challenge index. The auditor structures the challenge as: $chal = (fileID, Q, nonce, timestamp, AuditorSig)$, where $AuditorSig$ is the auditor's signature on the first four fields (for subsequent accountability).

The auditor sends $chal$ to the cloud server CS.

3) Proof Generation Phase

CS first checks whether the timestamp is within a reasonable range (to prevent delay attacks), and can optionally verify $AuditorSig$ (if CS has configured a verification policy). If it is illegal, the response is refused.

CS retrieves the corresponding ciphertext blocks $\{c_i | i \in Q\}$ from the storage according to the $fileID$ and the indexes in Q . If some blocks are lost or damaged, they are replaced with default values (such as all 0s) (but the subsequent proof will fail the verification).

For each challenged block c_i , CS calculates its lattice hash value $H(c_i)$, where H is the system-predefined lattice hash function (satisfying additive homomorphism). Using the additive homomorphism of lattice hash, CS calculates the aggregated hash value: $H_{agg} = H(c_1) + H(c_2) + \dots + H(c_i)$. Note: Since $H(c_1 || c_2 || \dots || c_k) = H(c_1) + \dots + H(c_k)$, the aggregated hash value is equivalent to the lattice hash value of the concatenated challenged blocks.

CS selects a random number $s \in Z_q$ (independently selected for each challenge) and calculates g^s . CS calculates $g^{s \cdot H_{agg}}$, where the exponentiation is performed on group G .

$proof = (g^s, g^{s \cdot H_{agg}}, Q, timestamp_response, CSSig)$, where $CSSig$ is CS's signature on the proof (for non-repudiation). CS sends proof back to the auditor.

4) Verification and On-Chain Phase

After receiving the proof, the auditor first verifies it locally, and then writes the result into the blockchain through the smart contract.

The auditor extracts g^r and the tags corresponding to the challenge index set $g^{r \cdot H(c_1)}, g^{r \cdot H(c_2)}, \dots, g^{r \cdot H(c_i)}$. Each tag is an element in group G , and group multiplication satisfies commutative and associative laws. Multiplying them gives $g^{r \cdot H(c_i)}$, resulting in $\{g^r, g^{r \cdot H(c_i)}\}$. Swap g^r and g^s to obtain $\{g^s, g^{r \cdot H(c_i)}\}$ and $\{g^r, g^{s \cdot H_{agg}}\}$. Use bilinear mapping to obtain $e(g^s, g^{r \cdot H(c_i)})$ and $e(g^r, g^{s \cdot H_{agg}})$. If they are equal, it proves that the data is complete, that is, the audit result is True. Otherwise, the audit result is False. According to the properties of bilinear mapping:

$$\begin{aligned} e(g^s, g^{r \cdot H(C)}) &= e(g, g)^{s \cdot r \cdot H(C)} \\ &= e(g^r, g^{s \cdot H(C)}) \\ &= e(g^r, g^{s \cdot H_{agg}}) \end{aligned} \quad (1)$$

Therefore, judging whether $e(g^s, g^{r \cdot H(C)}) = e(g^r, g^{s \cdot H_{agg}})$ is equivalent to judging whether $H(C)$ is equal to H_{agg} , that is, whether each data block is equal. If the data is incomplete, $e(g^s, g^{r \cdot H(C)}) \neq e(g^r, g^{s \cdot H_{agg}})$. Therefore, the auditor can use bilinear mapping to check whether the equation holds to verify the correctness of the stored data.

D. Smart Contract Evidence Storage

The auditor stores the result to the blockchain to achieve openness and transparency: Construct audit record: $R = (fileID, timestamp, Q_hash, proof_hash, result, AuditorSig)$, where $Q_hash = Hash(Q)$ to avoid leaking specific indexes, and $proof_hash = Hash(proof)$ for archiving.

Call smart contract: Send a transaction to the blockchain network, call the store function of the pre-deployed AuditLog contract, and pass in record.

Contract execution: After verifying the validity of the signature, the contract stores record in the historical array corresponding to the *fileID*.

Return transaction hash: The contract returns txHash, and the auditor can provide it to the vehicle as a voucher.

E. User Queries and Verification

- Quick Query: View historical audit results by calling the *fileID* via the blockchain.
- Independent Review: Download the encrypted file from the CS, recalculate all block labels, initiate a challenge and verify the results yourself, then compare them with the on-chain records. If any discrepancies are found, you can use the auditor's signature on the blockchain to trace responsibility.

IV. SIMULATION EXPERIMENT AND ANALYSIS

A. Security Analysis

- Verification correctness: If all challenged blocks are complete, $H(c'_i) = H(c_i)$, and the verification equation $e(g^s, g^{r \cdot H(C)}) = e(g^r, g^{s \cdot H_{agg}})$ holds. If any block is damaged, the collision resistance of the lattice hash ensures that the equation does not hold with overwhelming probability.
- Anti-replay attack: Each challenge contains a fresh random number s , and the proof generated by CS is bound to this challenge. Old proofs cannot pass the verification due to different random numbers.
- Audit accountability: The auditor signs the verification result and submits it to the smart contract. Once the result is on the chain, it cannot be tampered with. If the result is found to be inconsistent with the actual data state, the user can hold the auditor accountable by virtue of the on-chain signature, realizing "who audits, who is responsible".

B. Experimental Environment and Settings

This section tests the time overhead of each stage, including the initialization stage, challenge generation stage, proof generation stage, verification stage and on-chain evidence storage stage, and compares this scheme with the Merkle tree-based auditing scheme [10]. In the experiment, the size of a single data block is fixed at 128 KB, and the number of data blocks is 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000 respectively, corresponding to the file size from 12.8 MB to 128 MB. Each experiment is repeated 10 times and the average value is taken to reduce accidental errors. The experiment runs on a machine configured with Intel Core i5-9400 (2.90 GHz) and 16 GB memory, and the operating system is Windows 11. The code is developed based on Java 1.8 and JPBC cryptography library, and the Ethereum smart contract is deployed on the Ganache test network.

1) Initialization Phase

The time overhead of the initialization phase is mainly composed of two parts: the vehicle encrypting the original file and generating tags for all data blocks. As shown in Fig. 2, this scheme uses a symmetric encryption algorithm to encrypt the file. This process is linearly related to the file size but the overhead is very small after hardware acceleration. When generating tags, this scheme calculates a lattice hash value and an exponentiation operation for each ciphertext block. Since the exponentiation operation on the elliptic curve only takes milliseconds, and the lattice hash calculation is simple, the total time overhead is low. The comparison scheme is constructed based on asymmetric encryption or Merkle tree, and each block requires multiple modular exponentiations or hash iterations, resulting in large time overhead.

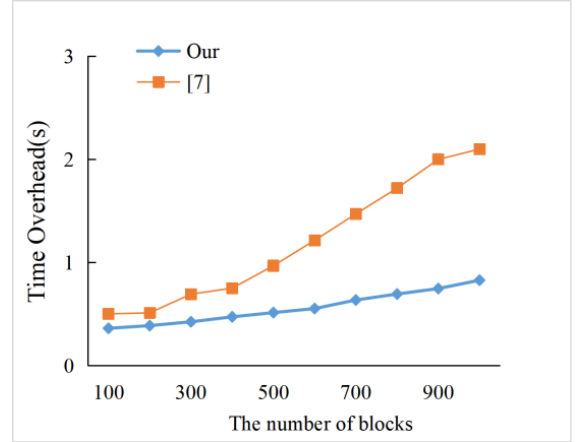


Fig. 2. Initialization Comparison

2) Challenge Phase and Proof Generation Phase

The challenge generation phase is executed by the auditor: randomly select c block indexes ($c=300$), and then calculate the aggregated tag of the challenged blocks. This operation only requires $c-1$ group multiplications, and the time consumption is linearly related to c but the absolute value is extremely small. When $c=300$, the time consumption of this scheme is about 2 milliseconds, which is basically the same as that of the comparison scheme and can be ignored.

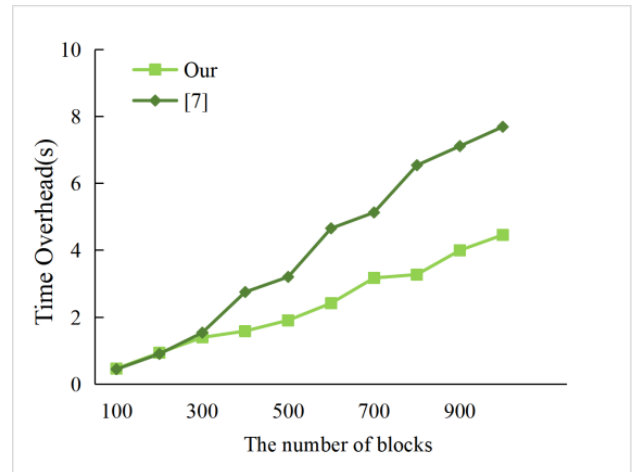


Fig. 3. Proof Generation Comparison

The proof generation phase is executed by the cloud server CS: after receiving the challenge, take out the corresponding c ciphertext blocks, calculate the lattice hash value of each block and sum them up, and then calculate g^S and g^{S-Hagg} . The main overhead of this process comes from c lattice hash calculations and two exponentiation operations. As shown in Fig. 3, the proof generation time of this scheme increases linearly with c , but the slope is very small. This comparison method requires extracting the attestation path from the Merkle tree and recalculating the root node, so it incurs greater overhead..

3) Verification Phase

The verification phase is executed by the auditor, and only needs to calculate two bilinear pairings. Since the number of pairings is fixed at 2, the verification time is independent of the number of challenged blocks c . As shown in Fig. 4. This comparison method requires multiple pairwise comparisons or hash operations, which is more time-consuming

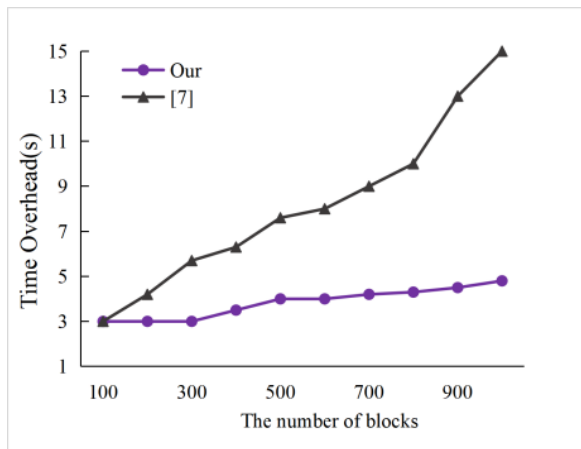


Fig. 4. Verification Phase Comparison

V. SUMMARY AND DISCUSSION

To address the deficiencies of existing schemes in terms of computational overhead, audit accountability, and transparent evidence storage, this paper proposes a lightweight solution based on lattice hash and smart contracts. Leveraging the additive homomorphism of lattice hash, the vehicle generates only one tag per ciphertext block. The cloud server aggregates the hash values of challenged blocks to produce a constant-size proof, while the auditor completes verification with just two bilinear pairings. The signed audit results are then stored on the blockchain via a smart contract, ensuring public traceability and accountability of the auditing process. Experimental results

demonstrate that this scheme provides an efficient, transparent, and accountable integrity auditing method for IoV cloud storage. Future work will focus on hardware implementation of lattice hash and support for dynamic data updates.

ACKNOWLEDGMENT

This work was supported by the Provincial First-class Undergraduate Major in Computer Science and Technology at Nanfang College Guangzhou, and also by Nanfang College, Guangzhou through grants: No. 2025126190006 for the project titled "The Spiritual Impact of Words: Design and Research on Musical Subtitle Translation", and No. S202512619025 for "Wenxin Zhizhi Chain: A Cross-Cultural Youth Text Resonance System Integrating AI Reasoning Chains and Traditional Culture".

REFERENCES

- [1] C. He, W. Wang, W. Jiang, Z. He, J. Wang, and X. Xie, "Security for the internet of vehicles with integration of sensing, communication, computing, and intelligence: a comprehensive survey," *Sensors*, vol. 25, no. 16, p. 5119, August 2025.
- [2] S. Ali, "Security and privacy in multi-cloud and hybrid cloud environments: challenges, strategies, and future directions," *Comput. Secur.*, vol. 157, p. 104599, October 2025.
- [3] R. Sharma, A. Mishra, and P. K. Singh, "A survey on enhancing cloud data security using blockchain technology," in *Proc. 2024 2nd Int. Conf. Cloud Computing, Technology and Security (IC3TS)*, Indore, India, December 2024, pp. 1-6.
- [4] G. Ateniese, R. Di Pietro, L. V. Mancini, and G. Tsudik, "Scalable and efficient provable data possession," in *Proc. 4th Int. Conf. Security and Privacy in Communication Networks (SecureComm)*, Istanbul, Turkey, 2008, pp. 1-10.
- [5] Juels and B. S. Kaliski Jr., "Pors: proofs of retrievability for large files," in *Proc. 14th ACM Conf. Computer and Communications Security (CCS)*, Alexandria, VA, USA, 2007, pp. 584-597.
- [6] C. Wang, Q. Wang, K. Ren, and W. Lou, "Privacy-preserving public auditing for data storage security in cloud computing," in *Proc. IEEE INFOCOM*, 2010, pp. 1-9.
- [7] Z. Liu, L. Ren, Y. Feng, S. Wang, and J. Wei, "Data integrity audit scheme based on quad Merkle tree and blockchain," *IEEE Access*, vol. 11, pp. 59263-59274, 2023.
- [8] M. Bellare and D. Micciancio, "A new paradigm for collision-free hashing: incrementality at reduced cost," in *Proc. EUROCRYPT*, 1997, pp. 163-192.
- [9] J. Li, Z. Song, Z. Zhang, Y. Li, and C. Cao, "In-vehicle digital forensics for connected and automated vehicles with public auditing," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 6368-6383, February 2024.
- [10] H. Tian, F. Peng, H. Quan, and C.-C. Chang, "Identity-based public auditing for cloud storage of internet-of-vehicles data," *ACM Trans. Internet Technol.*, vol. 22, no. 4, pp. 1-24, November 2022.